

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN



BÀI TẬP LỚN
HỌC MÁY CƠ BẢN

APPLYING MACHINE LEARNING TO PREDICT
STROKE

NGÀNH: **CÔNG NGHỆ PHẦN MỀM**

SINH VIÊN: **1. LÊ ĐỨC TUYỂN**

2. TẠ QUANG ĐẠI

LỚP: **12422TN**

NGƯỜI HƯỚNG DẪN: **TS. HOÀNG QUỐC VIỆT**

HƯNG YÊN – 2025

NHẬN XÉT

Nhận xét của giáo viên hướng dẫn

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

GIÁO VIÊN HƯỚNG DẪN

Hoàng Quốc Việt

MỤC LỤC

DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ	6
CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN.....	9
1.1 Bài toán	9
1.2 Trình bày dữ liệu bài toán	9
1.3 Tiền xử lý dữ liệu.....	11
1.4 Làm sạch dữ liệu	12
1.5 Trực quan hoá dữ liệu	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	13
2.1 RandomForestClassifier.....	13
2.1.1 Giới thiệu thuật toán RandomForestClassifier.....	13
2.1.2 Cấu trúc của mô hình RandomForest.....	13
2.1.3 Quá trình xây dựng mô hình RandomForest.....	13
2.1.4 Ưu điểm của RandomForest.....	13
2.1.5 Nhược điểm của RandomForest.....	14
2.1.6 Đánh giá hiệu suất.....	14
2.2 KNeighborsClassifier.....	14
2.2.1 Giới thiệu thuật toán KNeighborsClassifier.....	14
2.2.2 Cấu trúc của mô hình KNN.....	14
2.2.3 Quá trình xây dựng mô hình KNN.....	15
2.2.4 Ưu điểm của KNN	15
2.2.5 Nhược điểm của KNN.....	15
2.2.6 Đánh giá hiệu suất.....	15
2.3 Support Vector Classification (SVC)	15
2.3.1 Giới thiệu thuật toán SVC.....	15
2.3.2 Cấu trúc của mô hình SVC.....	16
2.3.3 Quá trình xây dựng mô hình SVC.....	16

2.3.4	Ưu điểm của SVC	16
2.3.5	Nhược điểm của SVC	16
2.3.6	Đánh giá hiệu suất	16
CHƯƠNG 3: GIẢI PHÁP, CÀI ĐẶT THỰC HIỆN		17
3.1	Mã nguồn tiền xử lý dữ liệu	17
3.2	Mã nguồn chức năng làm sạch dữ liệu	18
3.2.1	Xử lý giá trị thiếu	18
3.2.2	Xoá outlier	20
3.2.3	Ma trận tương quan	21
3.3	Mã nguồn chức năng Trực quan hoá dữ liệu	23
3.3.1	Biểu đồ thể hiện phân phối đặc trưng các phân loại	23
3.3.2	Biểu đồ thể hiện đếm số lần đột quy	24
3.3.3	Biểu đồ thể hiện các đặc trưng liên tục	25
3.3.4	Biểu đồ thể hiện mối tương quan giữa đột quy và các phần tử còn lại....	26
3.3.5	Biểu đồ thể hiện sự phân bố đột quy và không đột quy dành cho (age, avg_glucose_level)	27
3.3.6	Biểu đồ kiểm tra giá trị ngoại lai	27
3.3.7	Biểu đồ thể hiện sự trực quan hoá giá trị thiếu	28
3.4	Chuyển đổi dữ liệu	29
3.4.1	Xử lý dữ liệu chuỗi	29
3.4.2	Xử lý dữ liệu số	30
3.4.3	Xử lý dữ liệu mất cân bằng bằng SMOTE	31
3.5	Huấn luyện mô hình SKLEARN	31
3.5.1	Tìm kiếm mô hình	31
3.5.2	Điều chỉnh tham số	32
3.5.3	Mô hình RandomForestClassifier	35
3.5.4	Mô hình KNeighborsClassifier	36
3.5.5	Mô hình Support Vector Classification	37

CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG	38
KẾT LUẬN	39
TÀI LIỆU THAM KHẢO	41

DANH MỤC CÁC HÌNH VẼ, ĐỒ THỊ

Hình 1. 1	Hiển thị 5 hàng đầu và 5 hàng cuối của dữ liệu	10
Hình 1. 2	Thông tin tổng quan về dữ liệu.....	10
Hình 1. 3	Thông tin mô tả về DataFrame	11
Hình 3. 1	Import thư viện cần thiết, load, hiển thị 5 hàng đầu của DataFrame	17
Hình 3. 2	Kết quả hiển thị 5 hàng đầu của DataFrame	17
Hình 3. 3	Mã nguồn xóa cột “id”	17
Hình 3. 4	Thông tin về DataFrame	18
Hình 3. 5	Mã nguồn xử lý giá trị thiếu	18
Hình 3. 6	Hiển thị phần trăm giá trị thiếu của từng cột.....	19
Hình 3. 7	Loại bỏ tất cả các hàng trong DataFrame data có ít nhất một giá trị bị thiếu (NaN) và hiển thị 5 hàng cuối của DataFrame	19
Hình 3. 8	Kết quả hiển thị 5 hàng cuối của DataFrame	19
Hình 3. 9	Mã nguồn đếm số lượng giá trị thiếu (NaN) trong từng cột của DataFrame data.....	20
Hình 3. 10	Kết quả liệt kê số giá trị thiếu theo cột.....	20
Hình 3. 11	Mã nguồn xử lý dữ liệu ngoại lai	20
Hình 3. 12	Boxplot minh họa dữ liệu sạch.....	21
Hình 3. 13	Mã nguồn ma trận tương quan	21
Hình 3. 14	Kết quả Heatmap minh họa mức độ tương quan giữa các yếu tố	22
Hình 3. 15	Mã nguồn biểu đồ thể hiện phân phối đặc trưng các phân loại.....	23
Hình 3. 16	Kết quả biểu đồ bar thể hiện phân phối đặc trưng các phân loại	23
Hình 3. 17	Kết quả biểu đồ pie thể hiện phân phối đặc trưng các phân loại.....	24
Hình 3. 18	Mã nguồn biểu đồ thể hiện đếm số lần đột quy.....	24
Hình 3. 19	Kết quả biểu đồ thể hiện đếm số lần đột quy	25
Hình 3. 20	Mã nguồn biểu đồ thể hiện các đặc trưng liên tục.....	25
Hình 3. 21	Kết quả biểu đồ thể hiện các đặc trưng liên tục	25
Hình 3. 22	Mã nguồn biểu đồ thể hiện mối tương quan giữa đột quy và các phần tử còn lại	26

Hình 3. 23 Kết quả biểu đồ thể hiện mối tương quan giữa đột quy và các phần tử còn lại	26
Hình 3. 24 Mã nguồn biểu đồ thể hiện sự phân bố đột quy và không đột quy dành cho (age, avg_glucose_level)	27
Hình 3. 25 Kết quả biểu đồ thể hiện sự phân bố đột quy và không đột quy dành cho (age, avg_glucose_level)	27
Hình 3. 26 Mã nguồn biểu đồ kiểm tra giá trị ngoại lai	27
Hình 3. 27 Kết quả biểu đồ kiểm tra giá trị ngoại lai	28
Hình 3. 28 Mã nguồn biểu đồ thể hiện sự trực quan hoá giá trị thiếu	28
Hình 3. 29 Kết quả biểu đồ thể hiện sự trực quan hoá giá trị thiếu	28
Hình 3. 30 Import 1 số thư viện cần thiết	29
Hình 3. 31 Mã nguồn xử lý dữ liệu chuỗi	29
Hình 3. 32 Kết quả xử lý dữ liệu chuỗi	29
Hình 3. 33 Mã nguồn xoá cột phân loại gốc trong X_train và X_test	30
Hình 3. 34 Mã nguồn xử lý dữ liệu số	30
Hình 3. 35 Kết quả xử lý dữ liệu số	30
Hình 3. 36 Hiển thị số hàng và số cột có X_train và y_train	30
Hình 3. 37 Xử lý dữ liệu mất cân bằng bằng SMOTE	31
Hình 3. 38 Mã nguồn tìm kiếm mô hình dùng Lazypredict	31
Hình 3. 39 Kết quả tìm kiếm mô hình dùng Lazypredict	32
Hình 3. 40 Import các thư viện cần thiết	32
Hình 3. 41 Mã nguồn hàm tìm kiếm, điều chỉnh tham số tốt nhất	33
Hình 3. 42 Mã nguồn trực quan hoá, hiển thị kết quả sau khi chạy mô hình	34
Hình 3. 43 Mã nguồn mô hình RandomForestClassifier	35
Hình 3. 44 Kết quả mô hình RandomForestClassifier	35
Hình 3. 45 Mã nguồn mô hình KNeighborsClassifier	36
Hình 3. 46 Kết quả mô hình KNeighborsClassifier	36
Hình 3. 47 Mã nguồn mô hình Support Vector Classification	37
Hình 3. 48 Kết quả mô hình Support Vector Classification	37

Hình 4. 1 Sử dụng Streamlit để đưa bài toán lên giao diện web	38
---	----

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN

1.1 Bài toán

Bệnh đột quỵ là một trong những nguyên nhân hàng đầu dẫn đến tử vong và gây suy giảm chất lượng cuộc sống trên toàn cầu. Dự đoán sớm nguy cơ bị đột quỵ thông qua việc phân tích dữ liệu và áp dụng các thuật toán học máy có thể giúp phát hiện sớm, đẩy nhanh tiến trình điều trị, giảm tỷ lệ tử vong và hậu quả dài hạn.

Bài toán "Áp dụng các thuật toán học máy vào dự đoán bệnh đột quỵ" nhấn mạnh việc sử dụng dữ liệu lịch sử bao gồm các yếu tố nguy cơ như tuổi, huyết áp, tiểu sử bệnh lý, tật đồng máu, thói quen sinh hoạt, v.v. Từ đó, chúng ta sẽ xây dựng các mô hình dự báo sử dụng các kỹ thuật học máy như hồi quy logistic, các mô hình cây quyết định, rừng ngẫu nhiên, và máy vector hỗ trợ (SVM).

Phân tích này không chỉ nhằm đưa ra dự báo về nguy cơ đột quỵ cho từng cá nhân mà còn cung cấp thông tin quan trọng cho việc nâng cao nhận thức cộng đồng, tăng hiệu quả phòng ngừa và cải thiện chất lượng chăm sóc sức khỏe.

Bệnh đột quỵ thường xảy ra đột ngột, tuy nhiên, nguy cơ bị bệnh có thể được dự đoán dựa trên nhiều yếu tố nguy cơ. Việc phân tích dữ liệu bình thường sử dụng các bộ dữ liệu uy tín từ bệnh viện, trung tâm y tế, và các nghiên cứu khoa học. Thông qua phân tích, chúng ta có thể:

- Xác định những yếu tố nguy cơ chính dẫn đến đột quỵ.
- Đánh giá ảnh hưởng của từng yếu tố đến nguy cơ bệnh.
- Phát triển các mô hình dự báo có độ chính xác cao, giúp bác sĩ và bệnh nhân lên kế hoạch phòng ngừa hiệu quả.

Bài toán áp dụng các thuật toán học máy vào dự đoán bệnh đột quỵ không chỉ mang lại giá trị thực tiễn lớn trong lĩnh vực y tế mà còn là cơ hội để ứng dụng các kiến thức cơ bản về học máy vào một bài toán thực tế. Kết quả phân tích và mô hình dự đoán có thể hỗ trợ hiệu quả cho việc phòng ngừa và giảm thiểu tác động của bệnh đột quỵ trong cộng đồng.

1.2 Trình bày dữ liệu bài toán

Dữ liệu được lấy từ trang web này:

[Stroke Prediction Dataset | Kaggle](#)

Applying Machine Learning to predict Stroke

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

Hình 1. 1 Hiển thị 5 hàng đầu và 5 hàng cuối của dữ liệu

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 5110 non-null   object
1   age                   5110 non-null   float64
2   hypertension           5110 non-null   int64
3   heart_disease          5110 non-null   int64
4   ever_married           5110 non-null   object
5   work_type              5110 non-null   object
6   Residence_type         5110 non-null   object
7   avg_glucose_level      5110 non-null   float64
8   bmi                   4909 non-null   float64
9   smoking_status         5110 non-null   object
10  stroke                 5110 non-null   int64
dtypes: float64(3), int64(3), object(5)
memory usage: 439.3+ KB
```

Hình 1. 2 Thông tin tổng quan về dữ liệu

- Bộ dữ liệu được sử dụng trong bài toán này bao gồm các đặc trưng liên quan đến sức khỏe và nhân khẩu học của bệnh nhân, cụ thể như sau:

1. id: Mã định danh bệnh nhân (ID).
2. gender: Giới tính của bệnh nhân.
3. age: Tuổi của bệnh nhân.
4. hypertension: Tình trạng tăng huyết áp (0: Không, 1: Có).
5. heart_disease: Tình trạng bệnh tim (0: Không, 1: Có).
6. ever_married: Tình trạng hôn nhân (Yes/No).

7. work_type: Loại hình công việc (Private, Self-employed, Govt_job, Children, Never_worked).
 8. Residence_type: Loại nơi cư trú (Urban/Rural).
 9. avg_glucose_level: Mức đường huyết trung bình.
 10. bmi: Chỉ số khối cơ thể (Body Mass Index).
 11. smoking_status: Tình trạng hút thuốc (formerly smoked, never smoked, smokes, Unknown).
 12. stroke: Đột quy (0: Không, 1: Có).
- Dữ liệu bài toán là 1 file csv gồm 5110 rows $\times$ 12 columns
 - + Có 12 feature và mỗi feature có 5110 dữ liệu đầu vào
 - Sau khi mô tả dữ liệu ta có:

	Giá	Tổng đánh giá	Tuổi
count	599999.000000	599999.000000	599999.000000
mean	55.026482	249.933553	41.009558
std	25.972593	144.334560	13.566723
min	10.000297	0.000000	18.000000
25%	32.529910	125.000000	29.000000
50%	55.051762	250.000000	41.000000
75%	77.493413	375.000000	53.000000
max	99.999648	499.000000	64.000000

Hình 1. 3 Thông tin mô tả về DataFrame

- + Độ tuổi lớn nhất bị bệnh là 82 và độ tuổi nhỏ nhất là 0.08
- + Tình trạng tăng huyết áp MAX là 1 và MIN là 0
- + Tình trạng mắc bệnh tim MAX là 1 và MIN là 0
- + Mức đường huyết trung bình lớn nhất là 271.74 và nhỏ nhất là 55.12
- + Chỉ số khối cơ thể lớn nhất là 97.6 và nhỏ nhất là 10.3
- + Đột quy MAX là 1 và MIN là 0

1.3 Tiền xử lý dữ liệu

- a) Hiển thị 5 hàng đầu của dữ liệu
- b) Hiển thị số hàng, cột của dữ liệu
- c) Xoá cột id
- d) Hiển thị thông tin của dữ liệu
- e) Mô tả dữ liệu

- f) Hiển thị giá trị duy nhất trong mỗi cột

1.4 Làm sạch dữ liệu

- a) Xử lý giá trị thiếu
- b) Xoá outlier
- c) Ma trận tương quan

1.5 Trực quan hoá dữ liệu

- a) Biểu đồ thể hiện phân phối đặc trưng các phân loại
- b) Biểu đồ thể hiện đếm số lần đột quy
- c) Biểu đồ thể hiện các đặc trưng liên tục
- d) Biểu đồ thể hiện mối tương quan giữa đột quy và các phần tử còn lại
- e) Biểu đồ thể hiện sự phân bố đột quy và không đột quy dành cho (age, avg_glucose_level)
- f) Biểu đồ thể hiện sự Trực quan hoá giá trị thiếu

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 RandomForestClassifier

2.1.1 Giới thiệu thuật toán RandomForestClassifier

Thuật toán RandomForestClassifier là một phương pháp học máy phổ biến được sử dụng trong cả bài toán phân loại và hồi quy. Random Forest là một tập hợp các cây quyết định, trong đó mỗi cây được xây dựng dựa trên một tập dữ liệu mẫu ngẫu nhiên (bootstrapping) và một tập con các đặc trưng (feature subset) được chọn ngẫu nhiên tại mỗi nút phân chia. Kết quả cuối cùng được lấy bằng cách thực hiện trung bình (hồi quy) hoặc lấy đa số phiếu (phân loại) từ các cây thành phần.

2.1.2 Cấu trúc của mô hình RandomForest

Mô hình Random Forest bao gồm:

- **Cây quyết định (Decision Trees):** Các cây quyết định hoạt động như các mô hình thành phần.
- **Tập dữ liệu mẫu (Bootstrap Samples):** Mỗi cây được huấn luyện trên một tập con dữ liệu được lấy mẫu ngẫu nhiên có thay thế từ tập dữ liệu gốc.
- **Tập con đặc trưng (Feature Subset):** Tại mỗi nút của cây, một tập hợp ngẫu nhiên các đặc trưng được chọn để tìm phân chia tốt nhất.

2.1.3 Quá trình xây dựng mô hình RandomForest

- **Khởi tạo các cây quyết định:** Xác định số lượng cây trong rừng (hyperparameter: `n_estimators`).
- **Lấy mẫu dữ liệu:** Tạo tập dữ liệu mẫu ngẫu nhiên có thay thế cho từng cây.
- **Chọn tập con đặc trưng:** Tại mỗi nút, chọn ngẫu nhiên một tập con các đặc trưng.
- **Xây dựng cây:** Phân chia dữ liệu tại các nút theo tiêu chí như Gini Index hoặc Entropy.
- **Dự đoán:** Đối với phân loại, mỗi cây bỏ phiếu cho một nhãn, và nhãn cuối cùng được chọn dựa trên đa số phiếu.

2.1.4 Ưu điểm của RandomForest

- **Khả năng tổng quát hóa tốt:** Random Forest giảm thiểu vấn đề overfitting nhờ tích hợp nhiều cây.
- **Hiệu suất cao:** Hoạt động hiệu quả trên nhiều loại dữ liệu khác nhau, bao gồm cả dữ liệu không tuyến tính.

- **Đánh giá quan trọng đặc trưng:** Cung cấp thông tin về mức độ quan trọng của từng đặc trưng trong việc đưa ra dự đoán.
- **Độ chính xác cao:** Thường đạt độ chính xác cao hơn các mô hình đơn lẻ như Decision Tree.
- **Xử lý dữ liệu mất mát:** Random Forest có thể xử lý dữ liệu bị mất mát bằng cách ước tính giá trị bị thiếu.

2.1.5 Nhược điểm của RandomForest

- **Khó giải thích:** Mặc dù các cây quyết định riêng lẻ dễ hiểu, tập hợp Random Forest lại phức tạp hơn để diễn giải.
- **Chi phí tính toán cao:** Xây dựng nhiều cây quyết định đòi hỏi tài nguyên tính toán lớn.
- **Dễ bị bias:** Trong một số trường hợp, nếu dữ liệu không được chuẩn hóa tốt, mô hình có thể bị bias.

2.1.6 Đánh giá hiệu suất

Hiệu suất của RandomForestClassifier thường được đánh giá thông qua các chỉ số như Accuracy, Precision, Recall, và F1 Score. Đối với bài toán dự đoán bệnh đột quỵ, mô hình được kiểm tra dựa trên khả năng dự đoán đúng các trường hợp bệnh lý.

2.2 KNeighborsClassifier

2.2.1 Giới thiệu thuật toán KNeighborsClassifier

Thuật toán K-Nearest Neighbors (KNN) là một phương pháp học máy không tham số, được sử dụng phổ biến cho bài toán phân loại và hồi quy. KNN hoạt động dựa trên khoảng cách giữa một điểm dữ liệu mới và các điểm dữ liệu đã biết trong tập huấn luyện. Dữ liệu mới được phân loại dựa trên nhãn của k điểm lân cận gần nhất.

2.2.2 Cấu trúc của mô hình KNN

- **Tập huấn luyện:** Bao gồm các điểm dữ liệu đã được gán nhãn.
- **Khoảng cách:** Các khoảng cách như Euclidean, Manhattan, hoặc Minkowski được sử dụng để xác định mức độ gần gũi giữa các điểm.
- **Số lượng lân cận (k):** Đây là tham số chính quyết định số lượng điểm lân cận được sử dụng để phân loại dữ liệu mới.

2.2.3 Quá trình xây dựng mô hình KNN

- **Xác định giá trị k:** Thông thường k được chọn là số lẻ để tránh trường hợp đồng phiếu.
- **Tính khoảng cách:** Đo khoảng cách từ điểm cần dự đoán đến tất cả các điểm trong tập huấn luyện.
- **Chọn k lân cận gần nhất:** Lấy k điểm có khoảng cách nhỏ nhất.
- **Bỏ phiếu:** Xác định nhãn của điểm mới dựa trên nhãn của k điểm lân cận.

2.2.4 Ưu điểm của KNN

- **Đơn giản:** Thuật toán dễ hiểu và triển khai.
- **Linh hoạt:** Có thể sử dụng cho cả bài toán phân loại và hồi quy.
- **Không cần huấn luyện:** KNN không yêu cầu giai đoạn huấn luyện, chỉ cần lưu trữ dữ liệu.
- **Hiệu quả với dữ liệu nhỏ:** Hoạt động tốt với các tập dữ liệu có kích thước nhỏ.

2.2.5 Nhược điểm của KNN

- **Chi phí tính toán cao:** Khi kích thước dữ liệu lớn, việc tính khoảng cách đến tất cả các điểm trở nên rất tốn kém.
- **Nhạy cảm với dữ liệu nhiễu:** Các điểm nhiễu có thể ảnh hưởng lớn đến kết quả.
- **Phụ thuộc vào k:** Việc chọn giá trị k phù hợp là một thách thức.
- **Cần chuẩn hóa dữ liệu:** Khoảng cách giữa các điểm có thể bị ảnh hưởng bởi thang đo của các đặc trưng.

2.2.6 Đánh giá hiệu suất

Hiệu suất của KNN được đánh giá thông qua các độ đo như Accuracy, Precision, và Recall. Đối với bài toán dự đoán bệnh đột quỵ, KNN cần xử lý tốt sự phân bố không đồng đều giữa các lớp dữ liệu.

2.3 Support Vector Classification (SVC)

2.3.1 Giới thiệu thuật toán SVC

Thuật toán Support Vector Classification (SVC) là một phương pháp học máy mạnh mẽ, đặc biệt hiệu quả với các bài toán phân loại. SVC hoạt động bằng cách tìm một siêu phẳng tối ưu để phân chia các lớp dữ liệu, đảm bảo khoảng cách giữa siêu phẳng và các điểm gần nhất của mỗi lớp (margin) là lớn nhất.

2.3.2 Cấu trúc của mô hình SVC

- **Siêu phẳng (Hyperplane):** Là đường hoặc mặt phẳng phân chia không gian đặc trưng.
- **Margin:** Khoảng cách giữa siêu phẳng và điểm dữ liệu gần nhất từ mỗi lớp.
- **Hạt nhân (Kernel):** Chức năng ánh xạ dữ liệu vào không gian đặc trưng cao hơn để xử lý dữ liệu không tuyến tính.

2.3.3 Quá trình xây dựng mô hình SVC

- **Lựa chọn hạt nhân:** Xác định hàm kernel như Linear, Polynomial, RBF (Gaussian), hoặc Sigmoid.
- **Tìm siêu phẳng tối ưu:** Sử dụng thuật toán tối ưu hóa (như SMO) để xác định siêu phẳng phân chia dữ liệu.
- **Đánh giá:** Kiểm tra hiệu suất dựa trên dữ liệu kiểm thử.

2.3.4 Ưu điểm của SVC

- **Hiệu quả cao:** Là một trong những thuật toán tốt nhất cho các bài toán phân loại nhị phân.
- **Xử lý tốt dữ liệu không tuyến tính:** Nhờ vào các hàm kernel.
- **Tổng quát hóa tốt:** Hạn chế overfitting trong nhiều trường hợp.
- **Hiệu quả với dữ liệu nhỏ:** Hoạt động tốt ngay cả khi kích thước dữ liệu nhỏ.

2.3.5 Nhược điểm của SVC

- **Khó mở rộng:** Hiệu suất giảm khi kích thước dữ liệu tăng.
- **Chi phí tính toán cao:** Việc tìm siêu phẳng tối ưu yêu cầu tài nguyên lớn.
- **Nhạy cảm với tham số:** Cần lựa chọn cẩn thận tham số như C, gamma, và kernel.
- **Không phù hợp với dữ liệu mất cân bằng:** SVC có thể bị bias khi dữ liệu không cân bằng.

2.3.6 Đánh giá hiệu suất

Hiệu suất của SVC thường được đánh giá bằng các chỉ số như Accuracy, Precision, Recall, và F1 Score. Với bài toán dự đoán bệnh đột quỵ, SVC có thể tối ưu hóa phân loại nhờ vào khả năng xử lý dữ liệu không tuyến tính.

CHƯƠNG 3: GIẢI PHÁP, CÀI ĐẶT THỰC HIỆN

3.1 Mã nguồn tiền xử lý dữ liệu

1) Data Exploration ¶

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
from datetime import datetime
# Ignore all warnings
warnings.filterwarnings("ignore")

data = pd.read_csv("healthcare-dataset-stroke-data.csv")

data.head()
```

Hình 3. 1 Import thư viện cần thiết, load, hiển thị 5 hàng đầu của DataFrame

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Hình 3. 2 Kết quả hiển thị 5 hàng đầu của DataFrame

```
data.shape

# Drop column = 'id'
data.drop(columns='id', inplace=True)
```

Hình 3. 3 Mã nguồn xóa cột “id”

Đầu tiên em sử dụng hàm `df.info()` để kiểm tra các feature nào bị thiếu và có kiểu dữ liệu thế nào. Em hiển thị lại hàm `df` và dùng lại hàm `info` để kiểm tra xem dữ liệu. Và hình ảnh dưới là kết quả sau khi đã điền các dữ liệu:

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   gender                 5110 non-null  object 
1   age                   5110 non-null  float64
2   hypertension           5110 non-null  int64  
3   heart_disease         5110 non-null  int64  
4   ever_married          5110 non-null  object 
5   work_type              5110 non-null  object 
6   Residence_type        5110 non-null  object 
7   avg_glucose_level     5110 non-null  float64
8   bmi                   4909 non-null  float64
9   smoking_status        5110 non-null  object 
10  stroke                 5110 non-null  int64  
dtypes: float64(3), int64(3), object(5)
memory usage: 439.3+ KB
```

Hình 3. 4 Thông tin về DataFrame

3.2 Mã nguồn chức năng làm sạch dữ liệu

3.2.1 Xử lý giá trị thiếu

```
print((data.isna().sum()/len(data))*100)
```

Hình 3. 5 Mã nguồn xử lý giá trị thiếu

Câu lệnh này tính toán tỷ lệ phần trăm giá trị thiếu trong từng cột của DataFrame `data`.

- **`data.isna()`**: Xác định các giá trị thiếu (NaN) trong `data` và trả về DataFrame với giá trị True/False.
- **`.sum()`**: Tính tổng số True (giá trị thiếu) cho mỗi cột.
- **`len(data)`**: Số lượng hàng trong `data`.
- **`*100`**: Chuyển đổi tỷ lệ sang phần trăm.

Kết quả: Hiển thị phần trăm giá trị thiếu của từng cột, hỗ trợ xác định cần xử lý những cột nào.

```
gender          0.000000
age             0.000000
hypertension    0.000000
heart_disease   0.000000
ever_married    0.000000
work_type       0.000000
Residence_type  0.000000
avg_glucose_level 0.000000
bmi             3.933464
smoking_status  0.000000
stroke          0.000000
dtype: float64
```

Hình 3. 6 Hiển thị phần trăm giá trị thiếu của từng cột

```
data.dropna(inplace=True)
data.tail()
```

Hình 3. 7 Loại bỏ tất cả các hàng trong DataFrame data có ít nhất một giá trị bị thiếu (NaN) và hiển thị 5 hàng cuối của DataFrame

Câu lệnh này được sử dụng để xử lý giá trị thiếu và kiểm tra dữ liệu sau khi loại bỏ các hàng chứa giá trị thiếu:

data.dropna(inplace=True): Loại bỏ tất cả các hàng trong DataFrame data có ít nhất một giá trị bị thiếu (NaN). Tham số inplace=True đảm bảo thay đổi được thực hiện trực tiếp trên DataFrame data mà không tạo một bản sao.

data.tail(): Hiển thị 5 hàng cuối cùng của DataFrame data sau khi loại bỏ các giá trị thiếu.

Mục đích: Loại bỏ dữ liệu không đầy đủ và xem nhanh các dòng cuối để đảm bảo dữ liệu đã được xử lý đúng cách.

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
5104	Female	13.0	0	0	No	children	Rural	103.08	18.6	Unknown	0
5106	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

Hình 3. 8 Kết quả hiển thị 5 hàng cuối của DataFrame

```
data.isna().sum()
```

Hình 3. 9 Mã nguồn đếm số lượng giá trị thiếu (NaN) trong từng cột của DataFrame data.

- Đếm số lượng giá trị thiếu (NaN) trong từng cột của DataFrame data.
- Kết quả: Một bảng liệt kê số giá trị thiếu theo cột.

```
gender          0
age             0
hypertension    0
heart_disease   0
ever_married    0
work_type       0
Residence_type  0
avg_glucose_level 0
bmi             0
smoking_status  0
stroke          0
dtype: int64
```

Hình 3. 10 Kết quả liệt kê số giá trị thiếu theo cột.

3.2.2 Xoá outlier

```
# Hàm xử lý dữ liệu ngoại lai
def handle_outliers(df, columns):
    """
    Hàm xử lý dữ liệu ngoại lai bằng cách loại bỏ các giá trị ngoài khoảng IQR (Interquartile Range).

    Parameters:
    df (DataFrame): DataFrame chứa dữ liệu.
    columns (list): Danh sách các cột cần xử lý ngoại lai.

    Returns:
    DataFrame: DataFrame đã được xử lý ngoại lai.
    """
    for column in columns:
        # Tính IQR
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1

        # Tính giới hạn dưới và trên
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # Loại bỏ các giá trị ngoài phạm vi IQR
        df = df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]

    return df
```

Hình 3. 11 Mã nguồn xử lý dữ liệu ngoại lai

Hàm **handle_outliers** loại bỏ giá trị ngoại lai trong các cột bằng cách sử dụng **IQR** (Interquartile Range):

1) **Tính IQR:**

- Q1 (25%) và Q3 (75%) là các phân vị.
- $IQR = Q3 - Q1$.

2) **Giới hạn:**

- **Lower bound** = $Q1 - 1.5 * IQR$.
- **Upper bound** = $Q3 + 1.5 * IQR$.

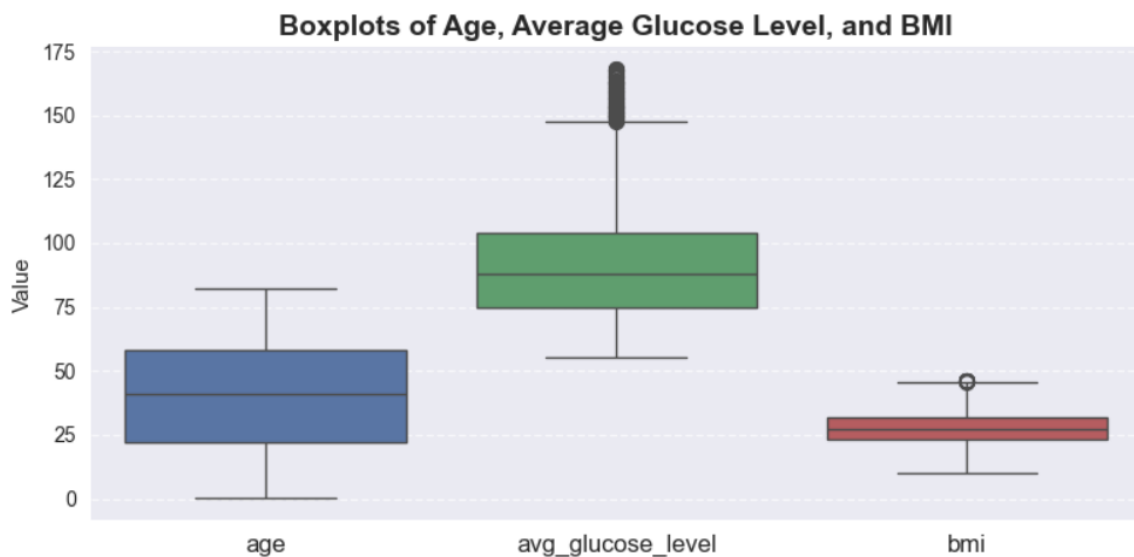
3) **Lọc dữ liệu:**

- Loại bỏ giá trị ngoài khoảng [lower_bound, upper_bound].

4) **boxplots(data_no_outliers):**

- Hiển thị boxplot của dữ liệu sau khi xử lý ngoại lai.

Kết quả: DataFrame data_no_outliers không còn ngoại lai, và boxplot minh họa dữ liệu sạch.



Hình 3. 12 Boxplot minh họa dữ liệu sạch.

3.2.3 Ma trận tương quan

```
# Define the columns to plot
num_plot = ['age', 'hypertension', 'heart_disease', 'avg_glucose_level', 'bmi']
# Set up the figure size
plt.figure(figsize=(8, 6))
# Plot the heatmap with correlations
sns.heatmap(data_no_outliers[num_plot].corr(), annot=True, fmt=".2f")
# Set the title and display it
plt.title("Understanding the factors influencing the target", fontsize=20)
plt.show()
```

Hình 3. 13 Mã nguồn ma trận tương quan

Biểu đồ Heatmap để thể hiện mối quan hệ tương quan giữa các cột số học trong dữ liệu đã xử lý ngoại lai:

1. **num_plot:**

- Danh sách các cột cần tính tương quan: ['age', 'hypertension', 'heart_disease', 'avg_glucose_level', 'bmi'].

2. **plt.figure(figsize=(8, 6)):**

- Thiết lập kích thước biểu đồ là 8x6 inch.

3. **sns.heatmap():**

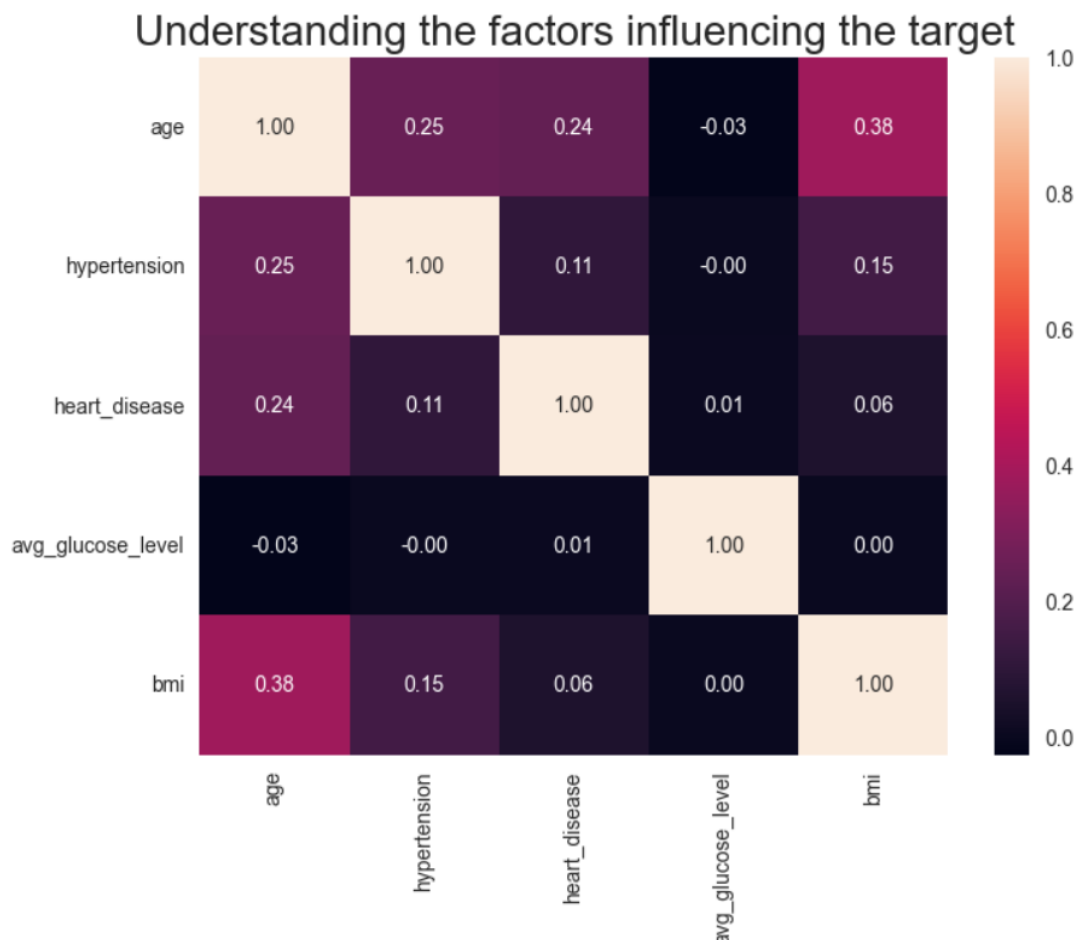
- Tính toán ma trận tương quan giữa các cột trong data_no_outliers.
- annot=True: Hiển thị giá trị tương quan trên heatmap.
- fmt=".2f": Định dạng giá trị tương quan đến 2 chữ số thập phân.

4. **plt.title():**

- Đặt tiêu đề: "Understanding the factors influencing the target".

5. **plt.show():** Hiển thị biểu đồ.

Kết quả: Heatmap minh họa mức độ tương quan giữa các yếu tố, giúp xác định yếu tố nào có tác động lớn hơn đến mục tiêu.



Hình 3. 14 Kết quả Heatmap minh họa mức độ tương quan giữa các yếu tố

3.3 Mã nguồn chức năng Trực quan hoá dữ liệu

3.3.1 Biểu đồ thể hiện phân phối đặc trưng các phân loại

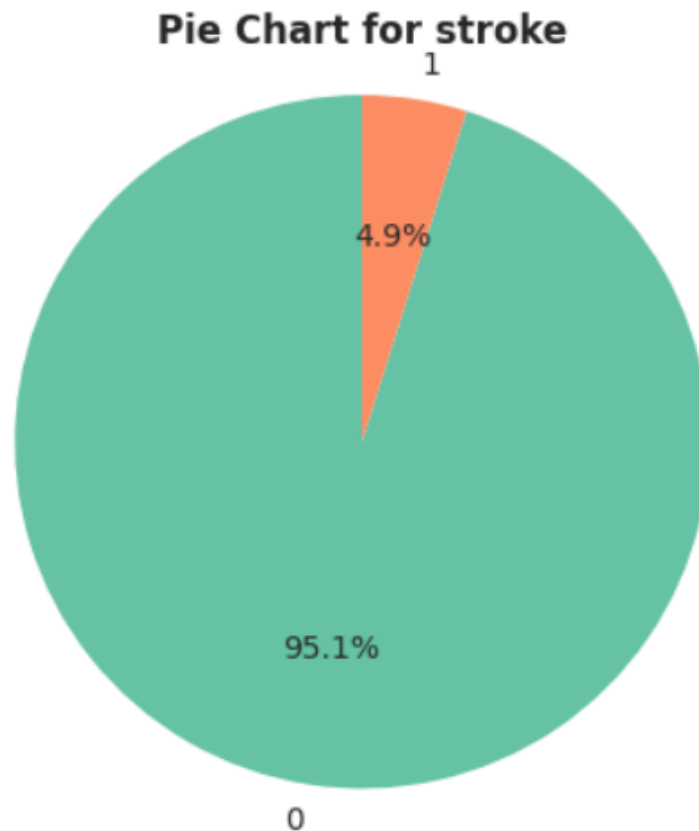
```
# Filter columns to process only categorical data
categorical_columns = [
    column for column in data.columns
    if data[column].dtype == 'object' or len(data[column].unique()) < 10
]
# Predefine the color palette to avoid generating it repeatedly
palette = sns.color_palette("Set2")

# Loop through categorical columns to create bar plots
for column in categorical_columns:
    plt.figure(figsize=(6, 4))
    value_counts = data[column].value_counts()
    # Use a subset of the predefined palette for colors
    colors = palette[:len(value_counts)]
    value_counts.plot(kind='bar', color=colors)
    plt.title(f"Bar Plot for {column}", fontweight='bold')
    plt.xlabel(column)
    plt.ylabel('Count')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

Hình 3. 15 Mã nguồn biểu đồ thể hiện phân phối đặc trưng các phân loại



Hình 3. 16 Kết quả biểu đồ bar thể hiện phân phối đặc trưng các phân loại



Hình 3. 17 Kết quả biểu đồ pie thể hiện phân phối đặc trưng các phân loại

3.3.2 Biểu đồ thể hiện đếm số lần đột quỵ

```
# Count stroke occurrences
stroke_counts = data['stroke'].value_counts()

# Define the explode and labels
explode = [0, 0.15]
labels = ['No Stroke (0)', 'Stroke (1)']

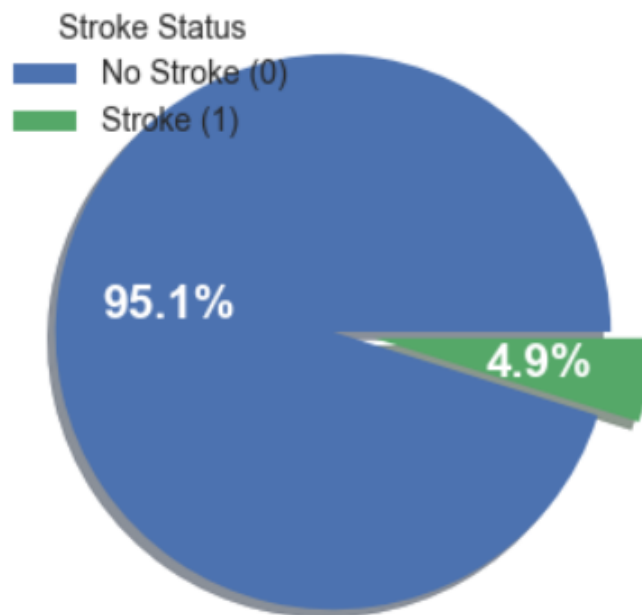
# Create a figure and axis
fig, ax = plt.subplots(figsize=(4, 4))

# Create the pie chart
ax.pie(stroke_counts, explode=explode, labels=labels, autopct='%1.1f%%',
       shadow=True, textprops=dict(color="w", weight='bold', fontsize=15))

# Add a Legend
plt.legend(title='Stroke Status', loc='upper left')

# Display the plot
plt.show()
```

Hình 3. 18 Mã nguồn biểu đồ thể hiện đếm số lần đột quỵ



Hình 3. 19 Kết quả biểu đồ thể hiện đếm số lần đột quỵ

3.3.3 Biểu đồ thể hiện các đặc trưng liên tục

```
import seaborn as sns
import matplotlib.pyplot as plt

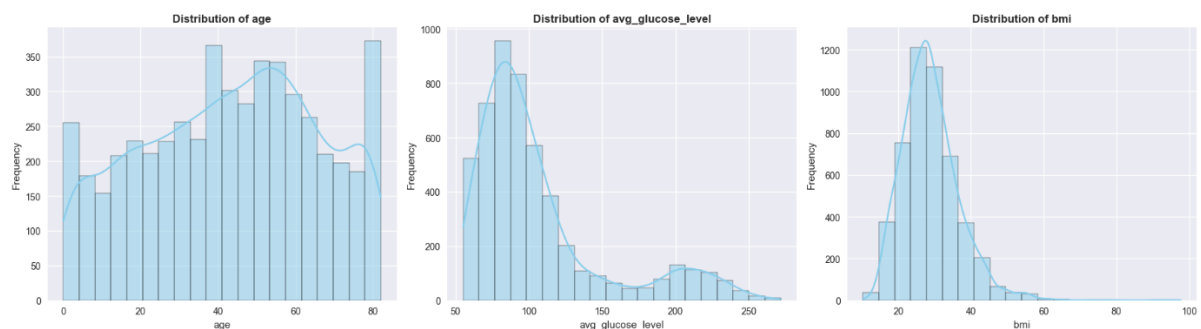
# Create a figure with subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# List of features to plot
features = ['age', 'avg_glucose_level', 'bmi']

# Loop through each feature to plot
for i, feature in enumerate(features):
    sns.histplot(data[feature], kde=True, ax=axes[i], color='skyblue', bins=20)
    axes[i].set_title(f'Distribution of {feature}', fontweight='bold')
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel('Frequency')

# Adjust layout to prevent overlap
plt.tight_layout()
plt.show()
```

Hình 3. 20 Mã nguồn biểu đồ thể hiện các đặc trưng liên tục



Hình 3. 21 Kết quả biểu đồ thể hiện các đặc trưng liên tục

3.3.4 Biểu đồ thể hiện mối tương quan giữa đột quỵ và các phần tử còn lại

```
# List of columns to include (excluding 'age', 'avg_glucose_level', and 'bmi')
stroke_relevant_columns = [col for col in data.columns if col not in ['age', 'avg_glucose_level', 'bmi', 'stroke']]

# Loop through each column to create bar plots for the correlation with 'stroke'
for column in stroke_relevant_columns:
    if column != 'stroke': # Skip the 'stroke' column itself
        plt.figure(figsize=(6, 4))

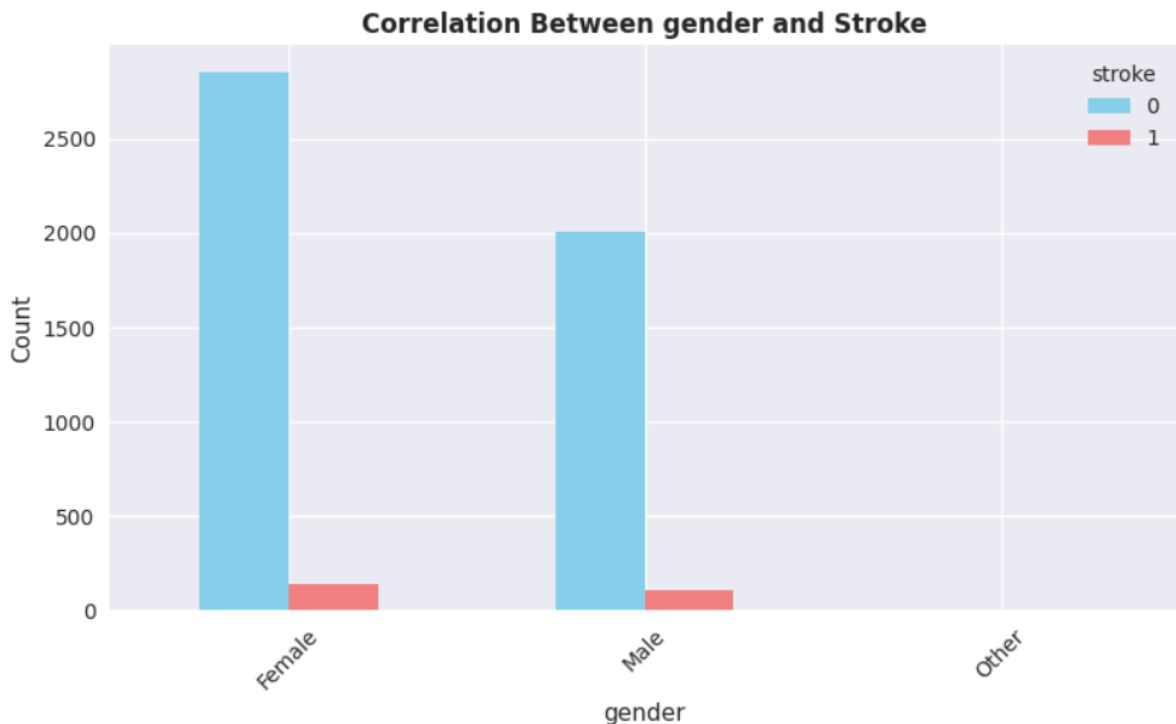
        # Group by 'stroke' and the current column, then count occurrences
        cross_tab = pd.crosstab(data[column], data['stroke'])

        # Plot the bar chart
        cross_tab.plot(kind='bar', stacked=False, color=['skyblue', 'lightcoral'], figsize=(8, 5))

        # Set the title and labels
        plt.title(f"Correlation Between {column} and Stroke", fontweight='bold')
        plt.xlabel(column)
        plt.ylabel('Count')
        plt.xticks(rotation=45)

        # Show the plot
        plt.tight_layout()
        plt.show()
```

Hình 3. 22 Mã nguồn biểu đồ thể hiện mối tương quan giữa đột quỵ và các phần tử còn lại



Hình 3. 23 Kết quả biểu đồ thể hiện mối tương quan giữa đột quỵ và các phần tử còn lại

3.3.5 Biểu đồ thể hiện sự phân bố đột quỵ và không đột quỵ dành cho (age, avg_glucose_level)

```
columns_to_plot = ['age', 'avg_glucose_level', 'bmi']

for col in columns_to_plot:
    plt.figure(figsize=(6, 6))

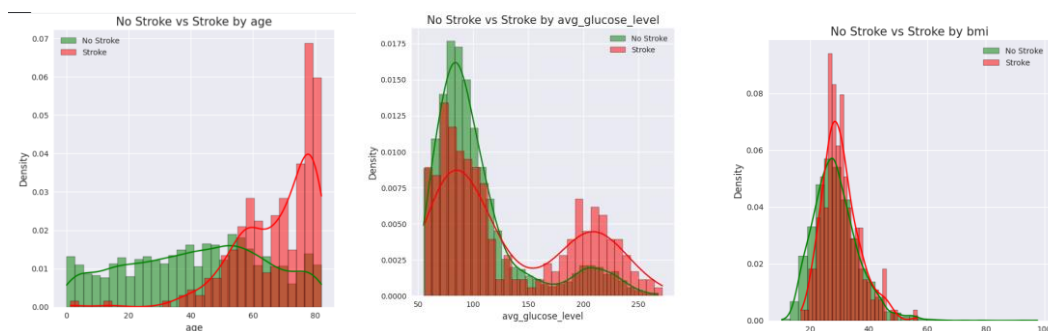
    # Plot distribution for 'stroke' == 0 (No Stroke)
    sns.histplot(data[data['stroke'] == 0][col], color='green', kde=True, label='No Stroke', bins=30, stat='density')

    # Plot distribution for 'stroke' == 1 (Stroke)
    sns.histplot(data[data['stroke'] == 1][col], color='red', kde=True, label='Stroke', bins=30, stat='density')

    # Add title and labels
    plt.title(f'No Stroke vs Stroke by {col}', fontsize=15)
    plt.xlabel(col, fontsize=12)
    plt.ylabel('Density', fontsize=12)
    plt.legend()

    # Show plot
    plt.tight_layout()
    plt.show()
```

Hình 3. 24 Mã nguồn biểu đồ thể hiện sự phân bố đột quỵ và không đột quỵ dành cho (age, avg_glucose_level)



Hình 3. 25 Kết quả biểu đồ thể hiện sự phân bố đột quỵ và không đột quỵ dành cho (age, avg_glucose_level)

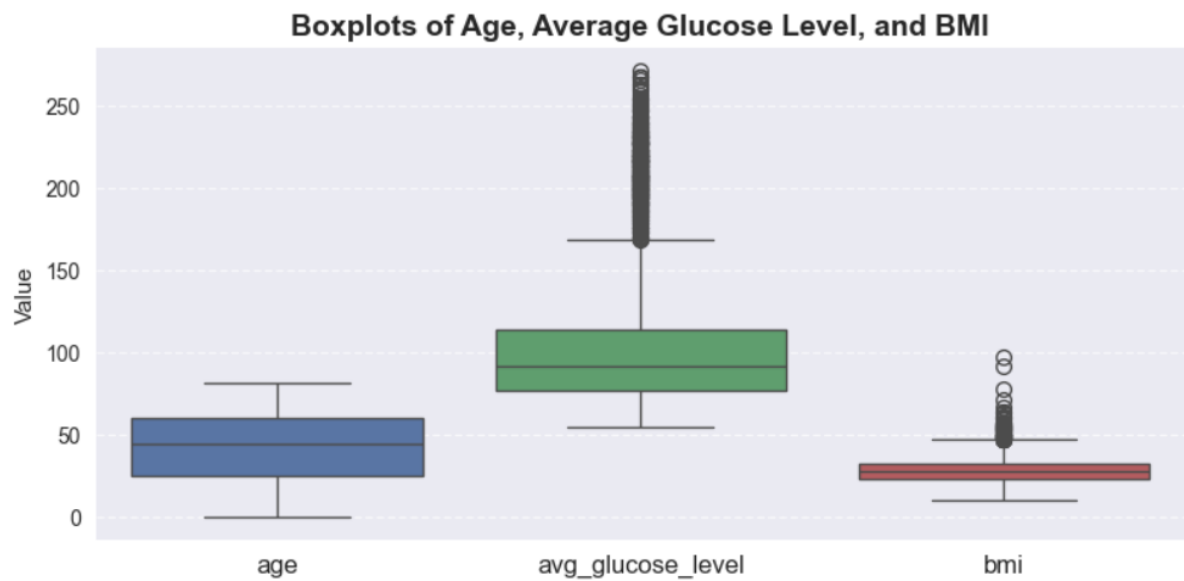
3.3.6 Biểu đồ kiểm tra giá trị ngoại lai

```
def boxplots(data):
    # Select the columns to visualize
    columns_to_plot = ['age', 'avg_glucose_level', 'bmi']

    # Create a boxplot for each column
    plt.figure(figsize=(8, 4))
    sns.boxplot(data=data[columns_to_plot])
    plt.title("Boxplots of Age, Average Glucose Level, and BMI", fontweight='bold', fontsize=14)
    plt.ylabel("Value")
    plt.xticks(ticks=range(len(columns_to_plot)), labels=columns_to_plot, fontsize=12)
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

boxplots(data)
```

Hình 3. 26 Mã nguồn biểu đồ kiểm tra giá trị ngoại lai

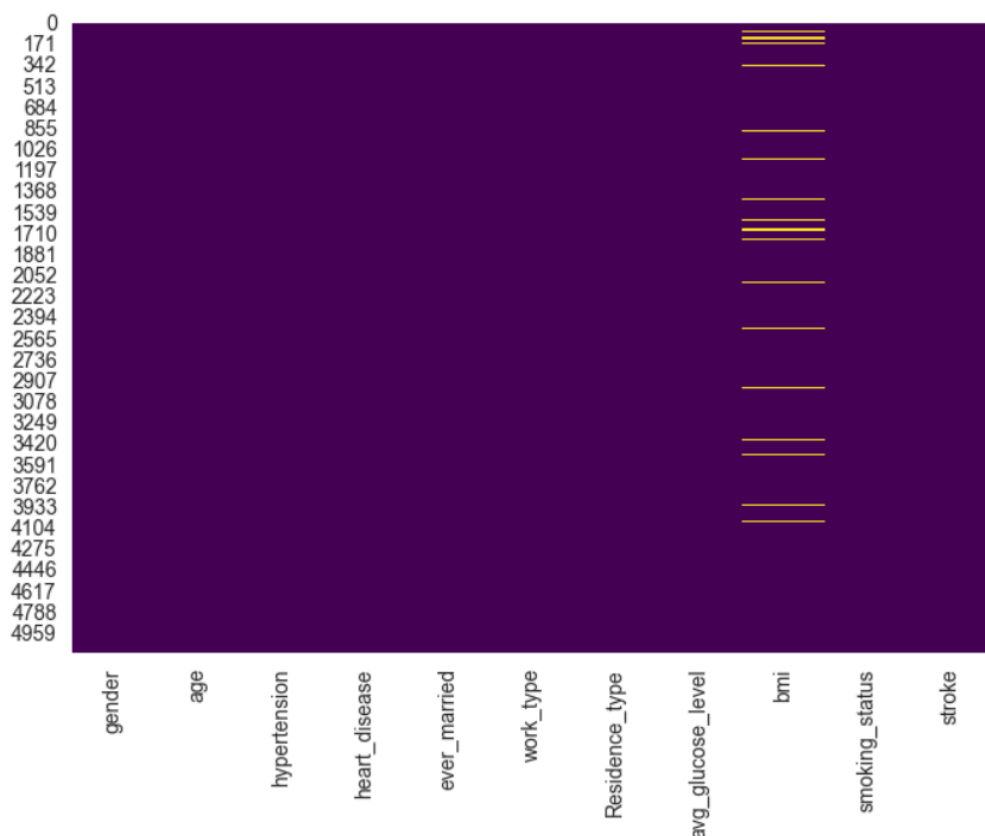


Hình 3. 27 Kết quả biểu đồ kiểm tra giá trị ngoại lai

3.3.7 Biểu đồ thể hiện sự trực quan hoá giá trị thiếu

```
sns.heatmap(data.isnull(), cmap='viridis', cbar=False)
```

Hình 3. 28 Mã nguồn biểu đồ thể hiện sự trực quan hoá giá trị thiếu



Hình 3. 29 Kết quả biểu đồ thể hiện sự trực quan hoá giá trị thiếu

3.4 Chuyển đổi dữ liệu

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.pipeline import Pipeline
```

Hình 3. 30 Import 1 số thư viện cần thiết

3.4.1 Xử lý dữ liệu chuỗi

```
# Giả sử dữ liệu đã được load vào data_no_outliers
# Tách các đặc trưng (X) và nhãn (y)
X = data_no_outliers.drop(columns=['stroke'])
y = data_no_outliers['stroke']

# 2. Chia dữ liệu thành train và test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

# 3. Các cột phân loại và số
categorical_columns = ['work_type', 'Residence_type', 'smoking_status', 'gender', 'ever_married']
numeric_columns = ['age', 'hypertension', 'heart_disease', 'avg_glucose_level', 'bmi']

# 4. One-Hot Encoding cho các cột phân loại
one_hot_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

# Áp dụng One-Hot Encoding trên tập train và test
X_train_encoded = pd.DataFrame(one_hot_encoder.fit_transform(X_train[categorical_columns]))
X_test_encoded = pd.DataFrame(one_hot_encoder.transform(X_test[categorical_columns]))

# Khôi phục tên cột cho dữ liệu sau One-Hot Encoding
X_train_encoded.columns = one_hot_encoder.get_feature_names_out(categorical_columns)
X_test_encoded.columns = one_hot_encoder.get_feature_names_out(categorical_columns)

X_train_encoded.head()
```

Hình 3. 31 Mã nguồn xử lý dữ liệu chuỗi

	work_type_Govt_job	work_type_Never_worked	work_type_Private	work_type_Self-employed	work_type_children	Residence_type_Rural	Residence_type_Urban	smoking_status_Ur
0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	
1	0.0	0.0	0.0	1.0	0.0	1.0	0.0	
2	0.0	0.0	0.0	1.0	0.0	1.0	0.0	
3	0.0	0.0	1.0	0.0	0.0	0.0	1.0	
4	0.0	0.0	1.0	0.0	0.0	1.0	0.0	

Hình 3. 32 Kết quả xử lý dữ liệu chuỗi

```
# Xóa cột phân loại gốc trong X_train và X_test
X_train = X_train.drop(columns=categorical_columns).reset_index(drop=True)
X_test = X_test.drop(columns=categorical_columns).reset_index(drop=True)
# Kết hợp dữ liệu sau One-Hot Encoding với các cột số ban đầu
X_train = pd.concat([X_train, X_train_encoded], axis=1)
X_test = pd.concat([X_test, X_test_encoded], axis=1)
```

Hình 3. 33 Mã nguồn xóa cột phân loại gốc trong X_{train} và X_{test}

3.4.2 Xử lý dữ liệu số

```
# 5. Chuẩn hóa các cột số bằng StandardScaler
scaler = StandardScaler()
# Áp dụng scaler lên tập train và test
X_train[numeric_columns] = scaler.fit_transform(X_train[numeric_columns])
X_test[numeric_columns] = scaler.transform(X_test[numeric_columns])
```

X_{train}

Hình 3. 34 Mã nguồn xử lý dữ liệu số

	age	hypertension	heart_disease	avg_glucose_level	bmi	work_type_Govt_job	work_type_Never_worked	work_type_Private	work_type_Self-employed	work_type
0	-0.020304	-0.267694	-0.192182	-0.913145	-0.111550	0.0	0.0	1.0	0.0	
1	-0.287787	-0.267694	-0.192182	-0.607360	0.622011	0.0	0.0	0.0	1.0	
2	0.782143	-0.267694	-0.192182	0.351976	1.265749	0.0	0.0	0.0	1.0	
3	-0.287787	-0.267694	-0.192182	-0.388626	0.023185	0.0	0.0	1.0	0.0	
4	0.470080	-0.267694	-0.192182	-0.217174	-0.425934	0.0	0.0	1.0	0.0	
...	...	...	...	...	...	...	...	...	...	...
2971	-0.020304	-0.267694	-0.192182	-0.946287	-0.680435	0.0	0.0	1.0	0.0	
2972	-1.001073	-0.267694	-0.192182	1.410737	-1.099613	0.0	0.0	1.0	0.0	
2973	0.113437	-0.267694	-0.192182	0.063424	0.726806	0.0	0.0	1.0	0.0	
2974	-0.956493	-0.267694	-0.192182	0.220294	-0.665465	0.0	0.0	1.0	0.0	
2975	-1.491458	-0.267694	-0.192182	-1.228652	-1.339144	0.0	0.0	0.0	1.0	

2976 rows × 21 columns

Hình 3. 35 Kết quả xử lý dữ liệu số

```
print(X_train.shape)
print(y_train.shape)
```

```
(2976, 21)
(2976,)
```

Hình 3. 36 Hiển thị số hàng và số cột có X_{train} và y_{train}

3.4.3 Xử lý dữ liệu mất cân bằng bằng SMOTE

```
# 6. Áp dụng SMOTE trên tập train (sau khi chuẩn hóa)
smote = SMOTE(sampling_strategy=0.8, random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print(X_train_smote.shape)
print(y_train_smote.shape)
print(X_test.shape)
print(y_test.shape)

(5185, 21)
(5185,)
(1276, 21)
(1276,)
```

Hình 3. 37 Xử lý dữ liệu mất cân bằng bằng SMOTE

3.5 Huấn luyện mô hình SKLEARN

3.5.1 Tìm kiếm mô hình

```
!pip install lazypredict
```

↕ Show hidden output

```
from lazypredict.Supervised import LazyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, make_scorer, confusion_matrix, roc_curve, auc
```

```
clf = LazyClassifier(verbose=0, ignore_warnings=True, custom_metric=None)
models, predictions = clf.fit(X_train_smote, X_test, y_train_smote, y_test)
```

97%|██████████| 28/29 [00:11<00:00, 3.63it/s]

```
[LightGBM] [Info] Number of positive: 2304, number of negative: 2881
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.003174 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3952
[LightGBM] [Info] Number of data points in the train set: 5185, number of used features: 19
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.444359 -> initscore=-0.223491
[LightGBM] [Info] Start training from score -0.223491
```

100%|██████████| 29/29 [00:12<00:00, 2.38it/s]

Hình 3. 38 Mã nguồn tìm kiếm mô hình dùng Lazypredict

```
models.head(15)
```

	Accuracy	Balanced Accuracy	ROC AUC	F1 Score	Time Taken
Model					
NearestCentroid	0.60	0.74	0.74	0.72	0.04
LinearSVC	0.77	0.74	0.74	0.84	0.59
LinearDiscriminantAnalysis	0.77	0.74	0.74	0.84	0.10
RidgeClassifierCV	0.77	0.74	0.74	0.84	0.17
RidgeClassifier	0.77	0.74	0.74	0.84	0.04
SGDClassifier	0.79	0.73	0.73	0.86	0.11
CalibratedClassifierCV	0.78	0.72	0.72	0.85	2.18
LogisticRegression	0.78	0.72	0.72	0.85	0.07
BernoulliNB	0.70	0.72	0.72	0.80	0.02
Perceptron	0.67	0.70	0.70	0.78	0.05
NuSVC	0.86	0.63	0.63	0.90	1.28
AdaBoostClassifier	0.86	0.63	0.63	0.90	0.36
PassiveAggressiveClassifier	0.68	0.63	0.63	0.78	0.04
SVC	0.90	0.59	0.59	0.92	0.67
KNeighborsClassifier	0.88	0.58	0.58	0.91	0.21

Hình 3. 39 Kết quả tìm kiếm mô hình dùng Lazypredict

3.5.2 Điều chỉnh tham số

```
# Thư viện sklearn
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC, SVC
```

Hình 3. 40 Import các thư viện cần thiết


```
def find_best_params(model, param_grid, X_train, y_train, X_test, y_test, scoring=make_scorer(recall_score), cv=5):
    """
    Find the best parameters for a model using GridSearchCV.

    Args:
        model: The model to optimize (e.g., LinearSVC(), RandomForestClassifier()).
        param_grid: The parameter grid to search (dict).
        X_train: Training data (features).
        y_train: Training data (labels).
        X_test: Test data (features).
        y_test: Test data (labels).
        scoring: Scoring function (default: recall_score).
        cv: Number of folds for cross-validation (default: 5).

    Returns:
        best_model: The best model with optimal parameters.
        results: Evaluation results on the test set.
    """
    # Initialize GridSearchCV
    grid_search = GridSearchCV(model, param_grid, scoring=scoring, cv=cv)

    # Train on the training set
    grid_search.fit(X_train, y_train)

    # Find the best parameters
    best_params = grid_search.best_params_
    print("Best parameters: ", best_params)

    # Get the best model
    best_model = grid_search.best_estimator_

    # Predict on the test set
    y_pred = best_model.predict(X_test)

    # Compute evaluation metrics
    results = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred)
    }

    # Print the results
    print("Test set accuracy: {:.2f}".format(results["accuracy"]))
    print("Test set precision: {:.2f}".format(results["precision"]))
    print("Test set recall: {:.2f}".format(results["recall"]))

    return best_model, results
```

Hình 3. 41 Mã nguồn hàm tìm kiếm, điều chỉnh tham số tốt nhất

Applying Machine Learning to predict Stroke

```
: def visualize_results(model, X_test, y_test, class_labels=None):
    """
    Visualize model results using a confusion matrix and ROC curve.

    Args:
        model: Trained model.
        X_test: Test data (features).
        y_test: Test data (labels).
        class_labels: List of class names (default: None).
                       If None, the default class indices will be used.

    Returns:
        None: The function directly displays the plots.
    """
    # Predict on the test set
    y_pred = model.predict(X_test)

    # Calculate prediction probabilities if the model supports it (e.g., predict_proba or decision_function)
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
    elif hasattr(model, "decision_function"):
        y_prob = model.decision_function(X_test)
    else:
        y_prob = None # Cannot compute ROC without probabilities

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(4, 4))
    sns.heatmap(
        cm,
        cmap='Blues',
        annot=True,
        fmt='d',
        linewidths=5,
        cbar=False,
        annot_kws={'fontsize': 15},
        yticklabels=class_labels if class_labels else [f"Class {i}" for i in range(cm.shape[0])],
        xticklabels=[f"Predicted {label}" for label in (class_labels if class_labels else [f"Class {i}" for i in range(cm.shape[1])])]
    )
    plt.xticks(rotation=0)
    plt.title("Confusion Matrix", fontsize=16)
    plt.show()

    # ROC Curve
    if y_prob is not None:
        false_positive_rate, true_positive_rate, thresholds = roc_curve(y_test, y_prob)
        roc_auc = auc(false_positive_rate, true_positive_rate)

        plt.figure(figsize=(4, 4))
        plt.plot(false_positive_rate, true_positive_rate, color='#b01717', label=f'AUC = {roc_auc:.3f}')
        plt.legend(loc='lower right')
        plt.plot([0, 1], [0, 1], linestyle='--', color='#174ab0')
        plt.axis('tight')
        plt.ylabel('True Positive Rate', fontsize=12)
        plt.xlabel('False Positive Rate', fontsize=12)
        plt.title('ROC Curve', fontsize=16)
        plt.show()
    else:
        print("Model does not support probability prediction (predict_proba or decision_function). ROC Curve cannot be plotted.")
```

Hình 3. 42 Mã nguồn trực quan hoá, hiển thị kết quả sau khi chạy mô hình

3.5.3 Mô hình RandomForestClassifier

RandomForestClassifier

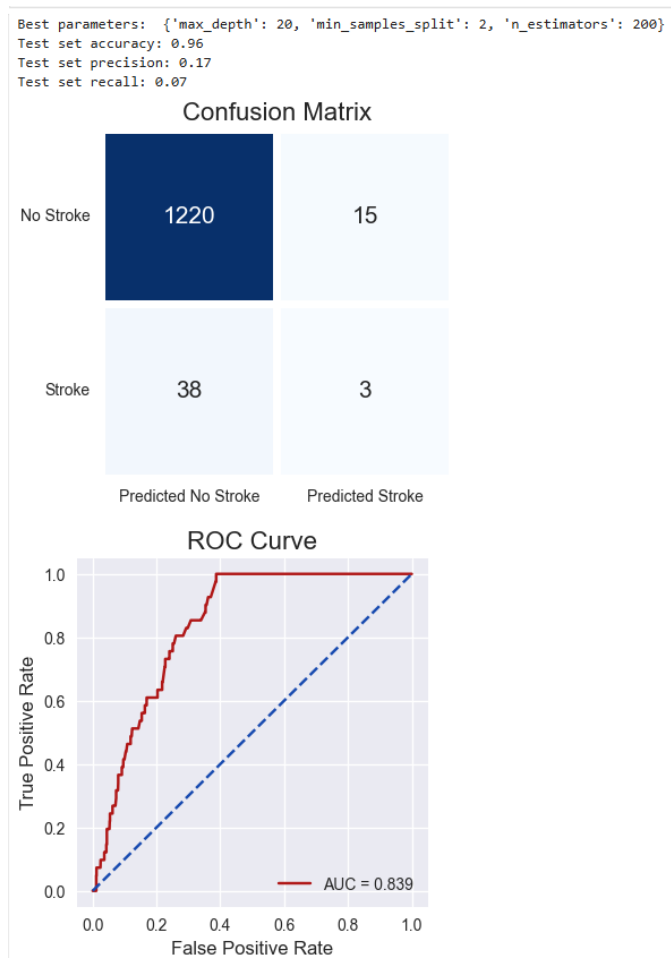
```
# Initialize RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)

# Define the parameter grid for RandomForestClassifier search
rf_param_grid = {
    'n_estimators': [50, 100, 200], # Number of trees in the forest
    'max_depth': [None, 10, 20, 30], # Maximum depth of the tree
    'min_samples_split': [2, 5, 10] # Minimum number of samples required to split a node
}

# Find the best parameters for RandomForestClassifier
rf_best_model, rf_test_results = find_best_params(
    model=rf_model,
    param_grid=rf_param_grid,
    X_train=X_train_smote,
    y_train=y_train_smote,
    X_test=X_test,
    y_test=y_test,
    scoring=make_scorer(recall_score),
    cv=5
)

# Visualize the results of RandomForestClassifier
visualize_results(rf_best_model, X_test, y_test, class_labels=['No Stroke', 'Stroke'])
```

Hình 3. 43 Mã nguồn mô hình RandomForestClassifier



Hình 3. 44 Kết quả mô hình RandomForestClassifier

3.5.4 Mô hình KNeighborsClassifier

KNeighborsClassifier

```
# Initialize KNeighborsClassifier
knn_model = KNeighborsClassifier()

# Define the parameter grid for KNN search
knn_param_grid = {
    'n_neighbors': [3, 5, 7],          # Number of nearest neighbors
    'weights': ['uniform', 'distance'], # Weighting method (uniform or distance-based)
    'metric': ['euclidean', 'manhattan', 'minkowski'] # Different distance metrics
}

# Find the best parameters for KNeighborsClassifier
knn_best_model, knn_test_results = find_best_params(
    model=knn_model,
    param_grid=knn_param_grid,
    X_train=X_train_smote,
    y_train=y_train_smote,
    X_test=X_test,
    y_test=y_test,
    scoring=make_scorer(recall_score),
    cv=5
)

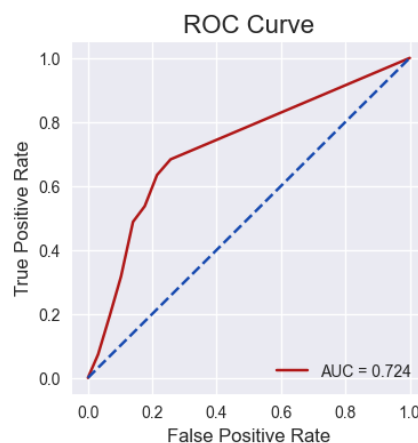
# Visualize the results of KNeighborsClassifier
visualize_results(knn_best_model, X_test, y_test, class_labels=['No Stroke', 'Stroke'])
```

Hình 3. 45 Mã nguồn mô hình KneighborsClassifier

Best parameters: {'metric': 'euclidean', 'n_neighbors': 7, 'weights': 'uniform'}
 Test set accuracy: 0.85
 Test set precision: 0.10
 Test set recall: 0.49

Confusion Matrix

	Predicted No Stroke	Predicted Stroke
No Stroke	1062	173
Stroke	21	20



Hình 3. 46 Kết quả mô hình KNeighborsClassifier

3.5.5 Mô hình Support Vector Classification

SVC

```
# Initialize SVM
svm_model = SVC(random_state=42)

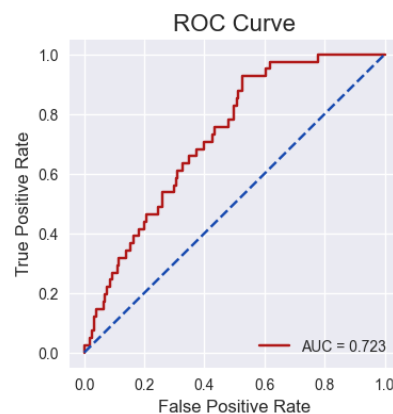
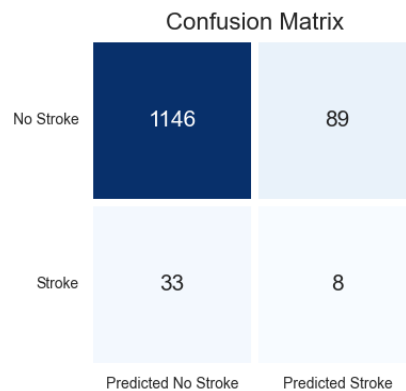
# Define the parameter grid for SVM search
svm_param_grid = {
    'C': [0.1, 1, 10],          # Regularization parameter
    'kernel': ['linear', 'rbf'], # Kernel types to use (linear or radial basis function)
    'gamma': ['scale', 'auto']  # Values for gamma (scale or auto)
}

# Find the best parameters for SVM
svm_best_model, svm_test_results = find_best_params(
    model=svm_model,
    param_grid=svm_param_grid,
    X_train=X_train_smote,
    y_train=y_train_smote,
    X_test=X_test,
    y_test=y_test,
    scoring=make_scorer(recall_score),
    cv=5
)

# Visualize the results of SVM
visualize_results(svm_best_model, X_test, y_test, class_labels=['No Stroke', 'Stroke'])
```

Hình 3. 47 Mã nguồn mô hình Support Vector Classification

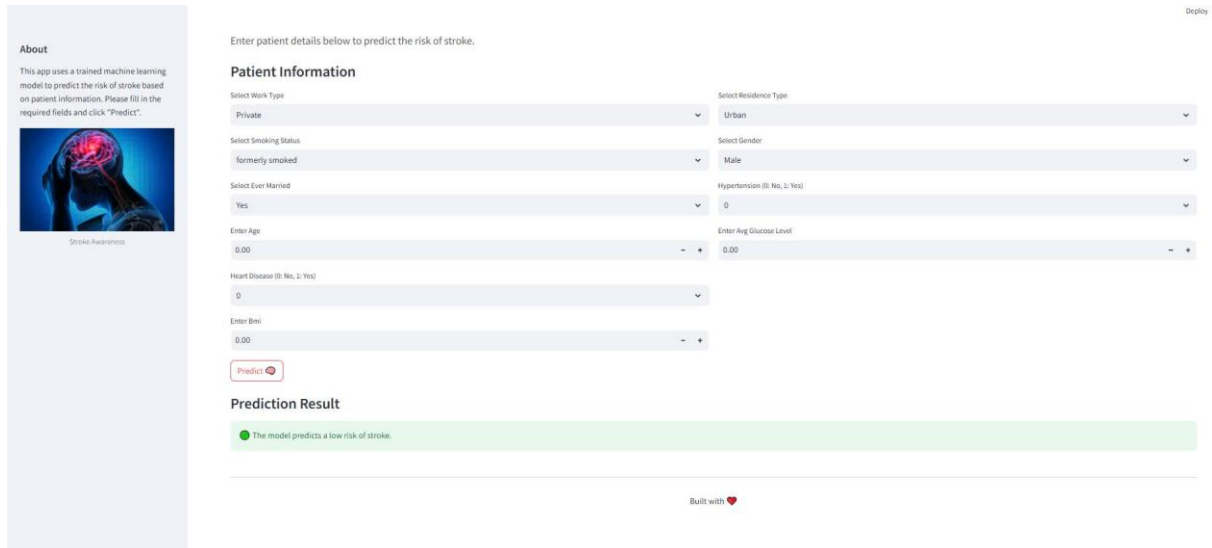
Best parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
 Test set accuracy: 0.90
 Test set precision: 0.08
 Test set recall: 0.20



Hình 3. 48 Kết quả mô hình Support Vector Classification

CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG

Bằng cách sử dụng Streamlit của Python em đã đưa bài toán dự đoán bệnh đột quỵ lên giao diện web.



The screenshot displays a web application interface for predicting stroke risk. On the left, an 'About' section explains the app's purpose and includes a brain scan image. The main area is titled 'Patient Information' and contains a form with various input fields: 'Select Work Type' (dropdown), 'Select Residence Type' (dropdown), 'Select Smoking Status' (dropdown), 'Select Gender' (dropdown), 'Select Ever Married' (dropdown), 'Hypertension (0, No, 1, Yes)' (dropdown), 'Enter Age' (text input), 'Enter Avg Glucose Level' (text input), 'Heart Disease (0, No, 1, Yes)' (dropdown), and 'Enter Bmi' (text input). A 'Predict' button is located below the form. The 'Prediction Result' section shows a green message: 'The model predicts a low risk of stroke.' The bottom of the page features a 'Built with' logo.

Hình 4. 1 Sử dụng Streamlit để đưa bài toán lên giao diện web

KẾT LUẬN

1. Kết quả đạt được trong kiến thức:

- **Hiểu cách hoạt động và tầm quan trọng của các mô hình RandomForestClassifier, KNeighborsClassifier, và Support Vector Classification:**
 - Hiểu cách RandomForestClassifier tổ hợp nhiều cây quyết định để tăng độ chính xác và khả năng tổng quát hóa.
 - Hiểu rõ nguyên tắc hoạt động của KNeighborsClassifier dựa trên người láng giềng gần nhất.
 - Hiểu biết về Support Vector Classification và cách nó tìm siêu phân cách tốt nhất trong không gian cao chiều.
- **Áp dụng các mô hình này vào bài toán dự đoán bệnh đột quỵ:**
 - Triển khai RandomForestClassifier, KNeighborsClassifier, và Support Vector Classification để phân loại nguy cơ bị đột quỵ dựa trên các đặc trưng như huyết áp, mỡ máu, tiểu sử bệnh tật.
 - Lựa chọn và tinh chỉnh các tham số quan trọng như số cây, k trong KNN, hay tham số kernel trong SVC để tối ưu hóa hiệu suất mô hình.
- **Xử lý, phân tích và trực quan hóa dữ liệu phù hợp với các mô hình:**
 - Thực hiện làm sạch dữ liệu, xử lý các giá trị thiếu, chuẩn hóa dữ liệu để đảm bảo tính nhất quán.
 - Trực quan hóa phân phối nguyên nhân để giúp nhân viên y tế hiểu tầm quan trọng của từng yếu tố.
- **Mô hình dự đoán bệnh đột quỵ có thể áp dụng trong hệ thống y tế:**
 - Các mô hình này có thể tích hợp vào hệ thống cảnh báo sớm trong bệnh viện hoặc các trung tâm y tế.
 - Cung cấp khả năng phân tích nhanh theo thời gian thực, tăng cơ hội chẩn đoán đột quỵ kịp thời.

2. Hạn chế của đề tài:

- **Biến động dữ liệu và yếu tố ngoài:**
 - Biến động nhanh chóng trong yếu tố nguy cơ như huyết áp hay cấp độ oxy có thể ảnh hưởng tới độ chính xác.

- **Cân bằng dữ liệu:**
 - Sự mất cân đối trong tập dữ liệu (các trường hợp bị đột quỵ thường rất thấp so với không bị) làm cho mô hình thiên về phân lớp chiếm đa số.
- **Giới hạn về tính tối ưu:**
 - Các mô hình như RandomForestClassifier và SVC phụ thuộc nhiều vào tham số tối ưu hóa, trong khi KNN yêu cầu sự cân nhắc cẩn thận với k.

3. Hướng phát triển của đề tài:

- **Tích hợp vào hệ thống y tế:**
 - Phát triển các ứng dụng trên nền tảng web và di động giúp bác sĩ và bệnh nhân giảm thiểu thời gian chẩn đoán.
- **Nâng cao tính giải thích:**
 - Sử dụng các kỹ thuật trực quan hóa và giải thích như LIME hoặc SHAP để tăng sự minh bạch và đáng tin cậy của mô hình.
- **Mở rộng tập dữ liệu:**
 - Thu thập thêm dữ liệu từ nhiều khu vực và nhóm tuổi để cải thiện tính tổng quát hóa.

TÀI LIỆU THAM KHẢO

1. Dữ liệu

[Stroke Prediction Dataset | Kaggle](#)

2. Tài liệu

- [1] Giáo trình Học máy cơ bản (Nhà xuất bản khoa học và kỹ thuật)
- [2] Các slide bài thuyết trình Học máy cơ bản – các nhóm trình bày.
- [3] Các slide bài giảng Học máy cơ bản – thầy TS. Hoàng Quốc Việt
- [4] Deep Learning - Tác giả: Ian Goodfellow, Yoshua Bengio, Aaron Courville
- [5] Python Machine Learning - Tác giả: Sebastian Raschka, Vahid Mirjalili
- [6] <https://www.coursera.org/learn/machine-learning>
- [7] <https://scikit-learn.org/stable/index.html>
- [8] <https://www.kaggle.com/>
- [9] <https://chatgpt.com/>

